BDY0101 · BASE DE DATOS NO RELACIONALES · EVALUACIÓN PARCIAL N°2 · FORMA B

QuickTrack: base de datos NoSQL en MongoDB Atlas integrada a una API REST

Informe técnico: configuración, carga, consultas, CRUD, integración y control de errores.

Integrantes: Franco Seguel · Martin Papic · Felipe Angel

BASE DE DATOS	APLICACIÓN	REPOSITORIO	EVIDENCIAS
MongoDB Atlas 8.0 · replica set de 3 nodos (AWS São Paulo)	Python 3.14 · PyMongo 4.19 · FastAPI 0.162	github.com/Francoseguel/QuickTrack	Ejecutadas el 26-10-2026 sobre el clúster real

500
documentos cargados en 5 colecciones, verificados contra los archivos de origen

−73 %
documentos examinados con el índice compuesto (100 → 27) según `explain()`

14 / 14
pruebas de integración y errores con el resultado esperado

82 ms
latencia mediana de una lectura a Atlas con un cliente reutilizado

Resumen ejecutivo

Se implementó la base de datos operativa de QuickTrack en un clúster de MongoDB Atlas y se integró con una API REST en Python (FastAPI + PyMongo). Los cinco conjuntos de datos del caso (500 documentos) se cargaron con un proceso ETL idempotente que además normaliza la referencia entre órdenes y clientes. Sobre esa base se desarrollaron consultas con filtros, proyecciones, agregaciones y ordenamientos; su eficiencia se midió con `explain()`, volumen de datos y latencia. También se validó el ciclo CRUD completo desde scripts y desde la API.

Las pruebas confirman lectura y escritura en tiempo real y respuestas de error controladas (400, 404, 422 y 500). El análisis detectó además dos puntos accionables: inconsistencias en los datos de origen (82 de 100 órdenes tienen un estado distinto al de su seguimiento) y una mejora de rendimiento en la API. Ambos se documentan con su solución propuesta.

Mapa de la pauta: dónde está cada apartado

Apartado de la pauta	Indicadores	Sección	Evidencia principal
1. Objetivos del proyecto	Transversal	1	Objetivo general y cuatro objetivos específicos alineados a IL2.1–IL2.4
2. Configuración de la base de datos	IE1 · IE2 · IE3	2	Clúster, colecciones y parámetros (partición, throughput, consistencia, credenciales); conectividad con 4 herramientas
3. Inserción de datos de prueba	IE1 · IE3 · IE7	3	ETL de 5 archivos; verificación archivo vs. base: 100 = 100 en las 5 colecciones
4. Desarrollo de consultas	IE4 · IE5 · IE6	4	5 consultas de negocio con resultados reales; evaluación con `explain()`, volumen y latencia
5. Operaciones CRUD completas	IE7 · IE8	5	Ciclo CRUD ejecutado; 14 controles de integridad (9 correctos, 5 inconsistencias con solución)
6. Integración con aplicación externa	IE9 · IE10	6	API REST con 6 operaciones; cada escritura vía HTTP se verifica al instante con PyMongo
7. Control de errores y validaciones	IE11	7	Respuestas 400/404/422/500 controladas, log de actividad y tiempo de proceso por request

Indicadores según la rúbrica de evaluación (IE1–IE11). Todas las cifras provienen de ejecuciones reales; las salidas de consola se reproducen sin editar en los recuadros oscuros de «Evidencia».

1 Objetivos del proyecto

Transversal

QuickTrack es una empresa de logística que ofrece entregas express a nivel nacional. Su sistema registra en una base de datos NoSQL las órdenes de entrega, el seguimiento de cada paquete, las rutas utilizadas y las valoraciones de los clientes. La gerencia necesita esa información disponible y consultable para optimizar rutas, identificar zonas críticas, medir la satisfacción del cliente y entender qué afecta el cumplimiento de plazos.

Objetivo general

Implementar y validar una base de datos NoSQL documental en MongoDB Atlas para los datos operativos de QuickTrack, integrada con una API REST en Python que permita consultar y modificar la información en tiempo real, con validación de entradas, control de errores y trazabilidad.

Objetivos específicos

#	Objetivo específico	Indicador de logro	Sección
OE1	Configurar un clúster de MongoDB Atlas con la base `quicktrack_db`, usuarios con permisos de lectura y escritura, y conexión segura mediante variables de entorno.	IL2.1	2
OE2	Desarrollar consultas con filtros, proyecciones, agregaciones y ordenamientos que respondan preguntas del negocio, y evaluar su eficiencia con datos medidos.	IL2.2	4
OE3	Cargar los datos del caso e implementar operaciones CRUD sobre órdenes y su seguimiento, validando los resultados e identificando errores e inconsistencias.	IL2.3	3 y 5
OE4	Integrar la base con una API REST que lea y escriba en tiempo real, con validación de entradas, manejo de errores y registro de actividad.	IL2.4	6 y 7

Alcance y supuestos

- **Datos:** los cinco archivos JSON de la Forma B (clientes, órdenes, rutas, seguimiento y valoraciones), con 100 documentos cada uno.
- **Entorno:** clúster compartido de MongoDB Atlas y API ejecutada localmente con Uvicorn (puerto 8000).
- **Evidencias:** todas las cifras provienen de ejecuciones del 26-10-2026. Las pruebas de escritura usaron identificadores de prueba (`orden_test_999`, `orden_api_901`) que se eliminaron al terminar; la base quedó con 100 documentos por colección.
- **Fuera de alcance:** visualizaciones en Power BI y despliegue de la API en un servidor público.

Resultado esperado para el negocio

Para la operación logística

Consultar en segundos el estado e historial de cualquier orden, actualizarlo desde una API y detectar las zonas con más cancelaciones.

Para la gerencia

Contar con indicadores por zona (cancelación, satisfacción, órdenes abiertas) y con un diagnóstico de la calidad de los datos registrados.

2 Configuración de la base de datos NoSQL

IE1 · IE2 · IE3

2.1 Tecnología seleccionada y justificación

La pauta propone Azure Cosmos DB (APIs Core SQL, Gremlin o Table) u otras tecnologías equivalentes, y recomienda MongoDB Atlas por su disponibilidad gratuita y su conexión externa. Se eligió **MongoDB Atlas**, cuyo modelo documental es el equivalente directo de la API Core (SQL) de Cosmos DB.

Alternativa	Modelo	Evaluación para QuickTrack
MongoDB Atlas ELEGIDA	Documental (BSON)	Los datos ya vienen en JSON. El historial de seguimiento se guarda embebido como arreglo, y `$lookup` + `$group` cubren el análisis por zona. Ofrece nivel gratuito, driver oficial para Python (PyMongo) y conexión por URI.
Cosmos DB · Core (SQL)	Documental	Equivalente conceptual, pero requiere suscripción de Azure y planificar RU/s; no aporta ventajas adicionales para el caso.
Cosmos DB · Gremlin	Grafo	Serviría para modelar la red de ciudades, pero las preguntas del caso son agregaciones por orden y zona, no recorridos de grafo.
Cosmos DB · Table	Clave-valor	No tiene agregaciones ni consultas compuestas ricas, así que no alcanza para el análisis requerido.

2.2 Arquitectura del clúster

Proveedor y región	AWS · `SA_EAST_1` (São Paulo), zona de disponibilidad `sae1-az3` (Sudamérica).
Topología	Replica set `atlas-8vgg5u-shard-0` de **3 nodos** (1 primario + 2 secundarios), con failover automático.
Versión del motor	MongoDB **8.0.34**
Base de datos	`quicktrack_db`: 5 colecciones, 500 documentos, ≈ 80 KB de datos y 8 índices.
Nivel de servicio	Clúster compartido de Atlas: no expone comandos administrativos (p. ej. `getDefaultRWConcern` responde *CommandNotFound*).
Conexión	URI `mongodb+srv://` con TLS: el driver descubre los 3 nodos mediante un registro DNS SRV.

Fuente: comandos `hello`, `server_info()` y `dbStats` ejecutados con PyMongo el 26-10-2026.

2.3 Colecciones (contenedores) y modelo de datos

Colección	Docs	Tam. prom.	_id (ejemplo)	Campos	Relación
`clientes`	100	135 B	`73` (entero)	nombre, correo, pais, fecha_registro	1:N con órdenes
`ordenes_entrega`	100	189 B	`"orden_2"`	cliente_id, cliente_num_id, ruta_id, fecha_envio, estado, medio_pago, total	N:1 con clientes y rutas
`rutas`	100	119 B	`"ruta_99"`	zona, ciudad_origen, ciudad_destino, distancia_km	1:N con órdenes
`seguimiento`	100	284 B	`"seg_2"`	orden_id, estado_actual, **historial[ ]** {estado, fecha}	1:1 con la orden (índice único)
`valoraciones`	100	90 B	`"val_2"`	orden_id, puntaje (1–5), comentario	1:1 con la orden

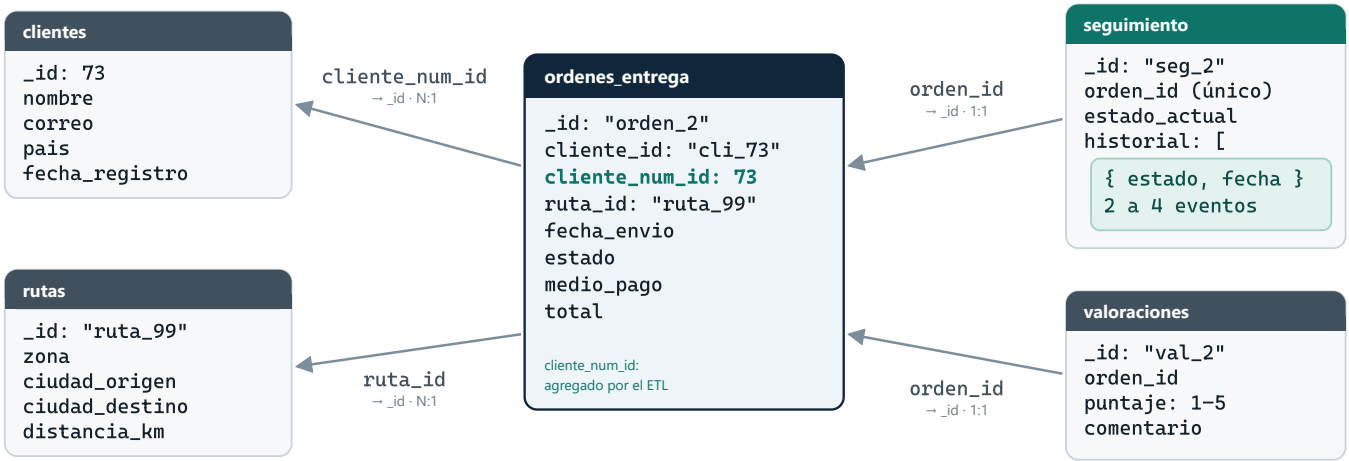


Figura 1. Modelo documental de `quicktrack_db`. Las flechas indican referencias entre colecciones; el historial de seguimiento va embebido.

Decisiones de modelado

- **Referencias** entre entidades con ciclo de vida propio (clientes, rutas, órdenes): así no se duplican datos que cambian por separado.
- **Embebido** para el historial de seguimiento: siempre se lee junto con su orden, crece acotado (de 2 a 4 eventos) y se actualiza de forma atómica con `$push`.
- **Relación 1:1** orden–seguimiento garantizada por el índice único `orden_id_1`.

Documento real de `seguimiento`

```
{ "_id": "seg_2", "orden_id": "orden_2",
  "estado_actual": "En preparación",
  "historial": [
    { "estado": "En preparación", "fecha": "2022-08-30T00:00:00" },
    { "estado": "Despachado",     "fecha": "2022-09-01T00:00:00" },
    { "estado": "En tránsito",    "fecha": "2022-09-03T00:00:00" },
    { "estado": "Entregado",      "fecha": "2022-09-05T00:00:00" } ] }
```

2.4 Parámetros esenciales y su validación

La pauta usa conceptos de Cosmos DB. La tabla muestra cada concepto, su equivalente en MongoDB Atlas, cómo quedó configurado en QuickTrack y cómo se validó.

Parámetro	Concepto en Cosmos DB	Configuración en MongoDB Atlas (QuickTrack)	Validación
Base y contenedores	Database + containers	Base `quicktrack_db` con 5 colecciones (una por entidad del caso), creadas por el ETL al insertar.	`list_collection_names()` → 5 colecciones **OK**
Clave de partición	Partition key obligatoria por contenedor	El replica set no está fragmentado, así que no requiere shard key: cada nodo guarda una copia completa. **Diseño para escalar:** `ordenes_entrega` → `{_id: "hashed"}` (alta cardinalidad, reparte escrituras); `seguimiento` y `valoraciones` → `{orden_id: "hashed"}` (la consulta por orden llega a un solo shard). Se descartan `estado` y `zona` por su baja cardinalidad (4 y 5 valores).	Acceso por orden ya indexado (`orden_id_1`) **DISEÑO**
Throughput	RU/s aprovisionadas o serverless	Atlas no usa RU/s: la capacidad depende del tier del clúster. Se midió el rendimiento real desde el equipo de desarrollo.	Ping: mediana 82 ms · lectura por `_id` : 82 ms · agregación por zona: 87 ms **MEDIDO**
Consistencia	5 niveles: Strong, Bounded staleness, Session, Consistent prefix, Eventual	Valores por defecto (la URI no los modifica): write concern implícito `w: "majority"` (MongoDB ≥ 5.0), read concern `local` y read preference `primary` . Cada cliente lee de inmediato lo que se escribió: equivale a **Session** en Cosmos DB.	Lo que escribe la API se lee al instante desde PyMongo (§6.5) **OK**
Credenciales	Claves de cuenta / RBAC	Usuarios de base de datos de Atlas (SCRAM) con rol `readWriteAnyDatabase` ; cada integrante invitado se conecta con su propio usuario. La URI vive en `.env` , que `.gitignore` excluye del repositorio; `.env.example` debe contener solo valores de ejemplo.	`connectionStatus` → rol `readWriteAnyDatabase` **OK**
Acceso de red	Firewall / VNet	Lista de acceso IP de Atlas y TLS obligatorio en la conexión `mongodb+srv` .	Conexión exitosa desde el equipo de desarrollo **OK**
Tiempo de espera	Timeouts del SDK	`serverSelectionTimeoutMS=5000` en `config/database.py` : si el clúster no responde, la operación falla a los 5 s en vez de bloquear la API.	Falla controlada con 500 (§7.3) **OK**

2.5 Configuración inicial

1. Crear el proyecto y el clúster en MongoDB Atlas (AWS, São Paulo).
2. Crear los usuarios de base de datos con permisos de lectura y escritura.
3. Autorizar las IP de los equipos en *Network Access*.
4. Copiar la URI `mongodb+srv://` y guardarla en `.env` (`MONGO_URI`, `DB_NAME`, `PORT`).
5. Instalar las dependencias de Python (Anexo A).
6. Validar con `python config/database.py`, que ejecuta un `ping` al clúster.

Evidencia 1 · prueba de conexión 26-10-2026

```
$ python config/database.py
 Conexión exitosa a la base de datos 'quicktrack_db'
en MongoDB Atlas.
```

Medición complementaria con PyMongo: el primer `ping` tardó 882 ms (incluye resolución DNS SRV, TLS y autenticación). La mediana de los 10 siguientes fue de **82 ms**.

2.6 Problemas encontrados y resueltos

Problema	Causa	Solución
Los archivos de datos usan dos formatos JSON distintos.	`clientes.json` es un arreglo; los otros cuatro son NDJSON (un documento por línea), y `json.load()` falla con NDJSON.	Dos cargadores: `cargar_coleccion_array()` y `cargar_coleccion_lineas()`.
El cruce órdenes–clientes devolvía 0 coincidencias.	`ordenes.cliente_id` es texto (`"cli_73"`) y `clientes._id` es entero (`73`).	Normalización en el ETL: nuevo campo `cliente_num_id`. Resultado: 100 de 100 órdenes se unen a su cliente (§5.3).
Cargar dos veces duplicaba los datos.	`insert_many()` agrega documentos sin revisar si ya existen.	`drop()` antes de insertar: la carga es idempotente.
Credenciales escritas en el código.	La URI incluye usuario y contraseña.	Archivo `.env` + `python-dotenv`; si falta `MONGO_URI`, se lanza un `ValueError` explícito.
Las dependencias fijadas no se instalaron en Python 3.14.	El proyecto se desarrolló en Python 3.13. `requirements.txt` fija pydantic 2.6.4 y pandas 2.2.1, que no publican paquetes precompilados para Python 3.14.	Se instalaron versiones compatibles (FastAPI 0.162, Pydantic 2.14, Pandas 3.0). Queda pendiente actualizar `requirements.txt`.

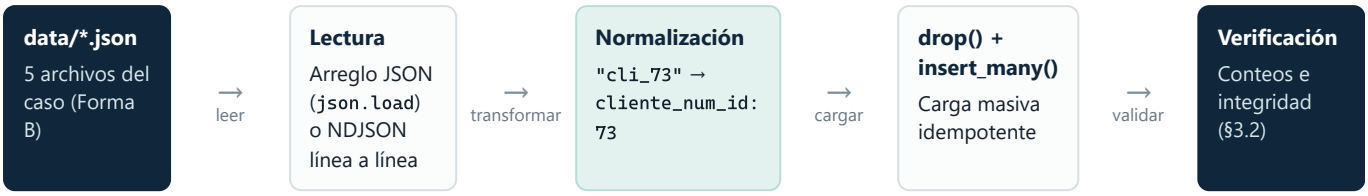
2.7 Conectividad con herramientas externas

Herramienta	Tipo	Prueba realizada	Resultado
PyMongo 4.19	SDK / driver	`ping` ×10, `hello`, consultas, CRUD y `explain()`	Estable · mediana 82 ms **OK**
FastAPI 0.162 + Uvicorn	Aplicación (API REST)	Lectura y escritura sobre Atlas a través de 6 operaciones HTTP	Todas responden **OK**
curl 8.18	Cliente HTTP externo	13 llamadas: CRUD completo, casos de error y analítica	13 / 13 con el código esperado **OK**
Swagger UI / OpenAPI	Explorador de la API	`/docs` y `/openapi.json` documentan las rutas automáticamente	5 rutas, 6 operaciones **OK**

3 Inserción de datos de prueba

IE1 · IE3 · IE7

3.1 Proceso de carga (ETL) — `scripts/cargar_atlas.py`



Archivo	Formato	Documentos	Colección destino	Transformación
`clientes.json`	Arreglo JSON	100	`clientes`	—
`ordenes_entrega.json`	NDJSON	100	`ordenes_entrega`	Agrega `cliente_num_id` (entero)
`rutas.json`	NDJSON	100	`rutas`	—
`seguimiento.json`	NDJSON	100	`seguimiento`	—
`valoraciones.json`	NDJSON	100	`valoraciones`	—

```
# scripts/cargar_atlas.py — lectura NDJSON, normalización y carga
for linea in f:
    linea_limpia = linea.strip()
    if linea_limpia:
        doc = json.loads(linea_limpia)
        # Normalización de cliente_id para ordenes_entrega ("cli_99" -> 99)
        if es_ordenes and isinstance(doc.get('cliente_id'), str) and doc['cliente_id'].startswith('cli_'):
            doc['cliente_num_id'] = int(doc['cliente_id'].replace('cli_', ''))
        documentos.append(doc)

db[nombre_coleccion].drop()                # carga idempotente
if documentos:
    db[nombre_coleccion].insert_many(documentos)
```

3.2 Verificación de la carga

Colección	Archivo	Base	Estado
`clientes`	100	100	OK
`ordenes_entrega`	100	100	OK
`rutas`	100	100	OK
`seguimiento`	100	100	OK
`valoraciones`	100	100	OK

Conteo de líneas de cada archivo contra `count_documents({})` en Atlas.

Controles adicionales tras la carga

- Tipos homogéneos en las 100 órdenes: `total` es *double*; `cliente_id` y `fecha_envio` son *string*.
- 0 órdenes con ruta inexistente y 0 seguimientos huérfanos.
- Exactamente 1 seguimiento y 1 valoración por orden.
- `fecha_envio` se guarda como texto ISO-8601. Sirve para ordenar y comparar, pero obliga a convertirla para cálculos de fechas (mejora en §4.5).

Documento cargado: `ordenes_entrega` → `orden_2`


```
{ "_id": "orden_2", "cliente_id": "cli_73", "ruta_id": "ruta_99", "fecha_envio": "2024-06-05T00:00:00",
  "estado": "Entregado", "medio_pago": "Tarjeta", "total": 165509.2, "cliente_num_id": 73 }
```

3.3 Inserciones unitarias de prueba

Además de la carga masiva, se probaron inserciones individuales por dos vías: el script CRUD (`orden_test_999`, §5.2) y la API (`POST /ordenes` con `orden_api_901`, §6.5). En ambos casos la orden y su seguimiento inicial quedaron en la base, se verificaron con una lectura y se eliminaron al terminar.

```
# scripts/crud_operations.py — CREATE: la orden y su primer evento de seguimiento
if db.ordenes_entrega.find_one({"_id": orden_id}):           # evita duplicados
    print(f" Advertencia: La orden '{orden_id}' ya existe."); return False
db.ordenes_entrega.insert_one({"_id": orden_id, "cliente_id": cliente_id, "ruta_id": ruta_id,
                               "fecha_envio": "2026-03-30T10:00:00", "estado": "En preparación",
                               "medio_pago": medio_pago, "total": float(total)})
db.seguimiento.insert_one({"_id": f"seg_{orden_id}", "orden_id": orden_id, "estado_actual": "En preparación",
                           "historial": [{"estado": "En preparación", "fecha": "2026-03-30T10:00:00"}]})
```

4 Desarrollo de consultas sobre la base de datos

IE4 · IE5 · IE6

Las consultas C1 a C3 están en `scripts/consultas_nosql.py`, y el endpoint `/analitica/rendimiento-zonas` de la API repite la agregación por zona. Para responder directamente las preguntas del caso se agregaron dos consultas complementarias (C4 y C5), ejecutadas con PyMongo para este informe.

4.1 Índices definidos

Colección	Índice	Tipo	Uso	Plan observado
`ordenes_entrega`	`{estado: 1, fecha_envio: -1}`	Compuesto	Filtrar por estado (C1) y listar órdenes de un estado por fecha, sin ordenar en memoria.	`IXSCAN`
`ordenes_entrega`	`{ruta_id: 1}`	Simple	Filtrar órdenes por ruta y hacer `$lookup` desde rutas hacia órdenes.	—
`seguimiento`	`{orden_id: 1}`	Único	Garantiza 1 seguimiento por orden y acelera el `$lookup` orden → seguimiento.	`EXPRESS_IXSCAN`

El `$lookup` de C2 (órdenes → rutas) usa el índice `_id` de `rutas`, que MongoDB crea automáticamente. El de C3 (valoraciones → órdenes) usó el índice `_id_` con 0 recorridos completos de colección.

4.2 Consultas implementadas en el repositorio

#	Requerimiento de negocio	Operadores y estructura
C1	¿Cuáles son las órdenes entregadas de mayor valor (más de $100.000)?	`find()` con filtro compuesto (igualdad + `$gt`), **proyección** de 5 campos, `sort(total, -1)` y `limit(5)`
C2	¿Cuántas órdenes y cuánto monto hay por zona y medio de pago?	Agregación: `$lookup` (rutas) → `$unwind` → `$group` con `$sum` / `$avg` → `$project` con `$round` → `$sort`
C3	¿Qué órdenes recibieron valoraciones críticas (≤ 2) y en qué estado están?	Agregación: `$match` con `$lte` → `$lookup` (órdenes) → `$unwind` → `$project` → `$sort`

```
# C1 — filtro compuesto + proyección + ordenamiento + límite
db.ordenes_entrega.find({"estado": "Entregado", "total": {"$gt": 100000}},
                        {"_id": 1, "cliente_id": 1, "medio_pago": 1, "total": 1, "fecha_envio": 1}
                       ).sort("total", -1).limit(5)
# C2 — agregación por zona y medio de pago
[{"$lookup": {"from": "rutas", "localField": "ruta_id", "foreignField": "_id", "as": "info_ruta"}},
 {"$unwind": "$info_ruta"},
 {"$group": {"_id": {"zona": "$info_ruta.zona", "medio_pago": "$medio_pago"},
             "total_ordenes": {"$sum": 1},
             "monto_acumulado": {"$sum": "$total"},
             "monto_promedio": {"$avg": "$total"}}},
 {"$project": {"_id": 0, "zona": "$_id.zona", "medio_pago": "$_id.medio_pago", "total_ordenes": 1,
               "monto_acumulado": {"$round": ["$monto_acumulado", 2]},
               "monto_promedio": {"$round": ["$monto_promedio", 2]}}},
 {"$sort": {"zona": 1, "monto_acumulado": -1}}]
# C3 — valoraciones críticas con el estado de su orden
[{"$match": {"puntaje": {"$lte": 2}}},
 {"$lookup": {"from": "ordenes_entrega", "localField": "orden_id", "foreignField": "_id", "as": "detalle_orden"}},
 {"$unwind": "$detalle_orden"},
 {"$project": {"_id": 0, "orden_id": 1, "puntaje": 1, "comentario": 1,
               "estado_orden": "$detalle_orden.estado", "monto_total": "$detalle_orden.total"}},
 {"$sort": {"puntaje": 1}}]
```

Evidencia 2 · salida de `python scripts/consultas_nosql.py` 26-10-2026

```
--- 1. Creación de Índices Estratégicos ---
 Índice creado en ordenes_entrega: estado_1_fecha_envio_-1
 Índice creado en ordenes_entrega: ruta_id_1
 Índice creado en seguimiento: orden_id_1

--- Consulta 1: Órdenes Entregadas con Total > $100.000 ---
  ID: orden_28 | Cliente: cli_11 | Pago: Transferencia | Total: $197,977.37
  ID: orden_26 | Cliente: cli_27 | Pago: Efectivo | Total: $182,075.53
  ID: orden_90 | Cliente: cli_4 | Pago: Efectivo | Total: $179,474.96
  ID: orden_97 | Cliente: cli_40 | Pago: Transferencia | Total: $176,107.92
  ID: orden_14 | Cliente: cli_71 | Pago: Tarjeta | Total: $168,699.70

--- Consulta 2: Rendimiento Logístico por Zona y Medio de Pago (Agregación) ---
  Zona: Centro | Pago: Efectivo | Órdenes: 8 | Total: $865,603.61
  Zona: Centro | Pago: Transferencia | Órdenes: 5 | Total: $460,064.32
  Zona: Centro | Pago: Tarjeta | Órdenes: 4 | Total: $415,379.19
  Zona: Norte | Pago: Transferencia | Órdenes: 10 | Total: $1,359,379.51
  Zona: Norte | Pago: Tarjeta | Órdenes: 9 | Total: $975,651.37
  Zona: Norte | Pago: Efectivo | Órdenes: 3 | Total: $363,067.78

--- Consulta 3: Análisis de Valoraciones Bajas (<= 2 puntos) ---
  Orden: orden_50 | Puntaje: 1/5 | Comentario: 'Tuvo retraso' | Estado: En preparación
  Orden: orden_100 | Puntaje: 1/5 | Comentario: 'Tuvo retraso' | Estado: En tránsito
  Orden: orden_99 | Puntaje: 1/5 | Comentario: 'Tuvo retraso' | Estado: Entregado
  Orden: orden_93 | Puntaje: 1/5 | Comentario: 'Muy buen servicio' | Estado: Cancelado
  Orden: orden_89 | Puntaje: 1/5 | Comentario: 'Excelente atención' | Estado: Cancelado

 Consultas ejecutadas correctamente.
```

4.3 Consultas complementarias: zonas críticas y satisfacción

C4 y C5 responden directamente dos preguntas del caso: *¿dónde está la insatisfacción?* y *¿qué zonas cumplen peor?*. Ambas combinan tres colecciones con `$lookup` y calculan indicadores con `$cond`.

```
# C4 — satisfacción por zona (valoraciones → órdenes → rutas)
db.valoraciones.aggregate([
  {"$lookup": {"from": "ordenes_entrega", "localField": "orden_id", "foreignField": "_id", "as": "o"}},
  {"$unwind": "$o"},
  {"$lookup": {"from": "rutas", "localField": "o.ruta_id", "foreignField": "_id", "as": "r"}},
  {"$unwind": "$r"},
  {"$group": {"_id": "$r.zona", "valoraciones": {"$sum": 1}, "puntaje_prom": {"$avg": "$puntaje"},
              "criticas": {"$sum": {"$cond": [{"$lte": ["$puntaje", 2]}, 1, 0]}}}},
  {"$sort": {"puntaje_prom": 1}}])
# C5 — cumplimiento por zona: canceladas, entregadas y abiertas
db.ordenes_entrega.aggregate([
  {"$lookup": {"from": "rutas", "localField": "ruta_id", "foreignField": "_id", "as": "r"}},
  {"$unwind": "$r"},
  {"$group": {"_id": "$r.zona", "ordenes": {"$sum": 1},
              "canceladas": {"$sum": {"$cond": [{"$eq": ["$estado", "Cancelado"]}, 1, 0]}},
              "entregadas": {"$sum": {"$cond": [{"$eq": ["$estado", "Entregado"]}, 1, 0]}},
              "abiertas":   {"$sum": {"$cond": [{"$in": ["$estado", ["En preparación", "En tránsito"]]}, 1, 0]}}}},
  {"$project": {"zona": "$_id", "ordenes": 1, "canceladas": 1, "entregadas": 1, "abiertas": 1,
                "pct_cancelacion": {"$round": [{"$multiply": [{"$divide": ["$canceladas", "$ordenes"]}, 100]}, 1]}}},
  {"$sort": {"pct_cancelacion": -1}}])
```

Zona	Órdenes	Canceladas	% cancelación	Abiertas	Puntaje prom.	% valoraciones ≤ 2	Distancia prom. rutas
Centro ZONA CRÍTICA	17	8	**47,1 %**	4	**2,71**	**47,1 %**	1.468 km
Norte	22	8	36,4 %	10	3,00	36,4 %	983 km
Sur	13	3	23,1 %	8	3,00	46,2 %	1.047 km
Poniente	22	2	9,1 %	11	3,14	31,8 %	1.283 km
Oriente	26	2	7,7 %	17	3,35	30,8 %	985 km
Total	**100**	**23**	**23,0 %**	**50**	**3,07**	**37,0 %**	—

Resultados de C4 y C5 (26-10-2026). «Abiertas» = órdenes en preparación o en tránsito. La distancia promedio considera todas las rutas de cada zona (`$avg` de `distancia_km`).

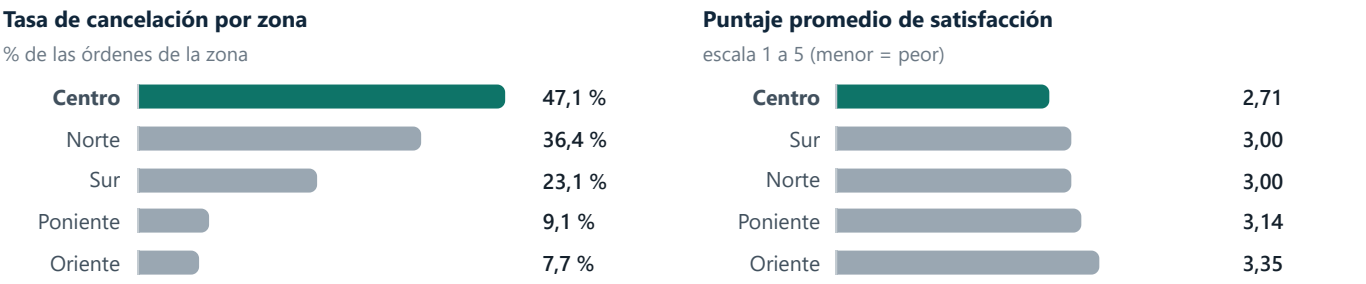


Figura 2. Centro (destacada) encabeza ambos indicadores negativos. Las barras de puntaje usan una escala de 0 a 5.

Hallazgo: Centro es la zona crítica

Centro tiene la mayor tasa de cancelación (47,1 %), el peor puntaje promedio (2,71 de 5) y la mayor distancia promedio de sus rutas (1.468 km). Oriente es el caso opuesto en cancelación (7,7 %), pero acumula la mayor cantidad de órdenes abiertas (17 de 26). En total, 50 de las 100 órdenes siguen en preparación o en tránsito, aunque sus fechas de envío son de 2022 a 2024.

Limitación de los datos para medir plazos

Las 33 órdenes cuyo historial llega a «Entregado» demoran siempre exactamente 6 días (un evento cada 2 días), así que los datos no permiten medir retrasos reales. Además, el conjunto no incluye campos que el caso menciona: fecha de entrega, motivo de retraso, tiempo estimado, tipo de transporte y clima. Se recomienda incorporarlos para construir indicadores de cumplimiento de plazos.

4.4 Consultas de modificación

Operación	Filtro	Operadores de modificación	Validación
Cambio de estado de una orden	`{"_id": orden_id}` / `{"orden_id": orden_id}`	`$set` del estado en la orden, y `$set` de `estado_actual` + `$push` de un evento al historial en el seguimiento	`matched_count`; el historial pasa de 1 a 2 eventos (§6.5)
Eliminación en cascada	`{"orden_id": orden_id}` y `{"_id": orden_id}`	`delete_many` en valoraciones y seguimiento, y `delete_one` en la orden	`deleted_count`; 0 documentos residuales
Corrección masiva **PROPUESTA**	`{}` (todos)	`update_many` con pipeline: `estado_actual` = último evento de `historial`	Corregiría los 72 seguimientos inconsistentes (§5.3)

4.5 Evaluación de eficiencia

Método: (1) `explain("executionStats")` de cada consulta con su índice, y de la misma consulta forzando un recorrido completo con `hint({"$natural": 1})`; (2) tamaño BSON de lo retornado, con y sin proyección; (3) latencia medida desde Python: 30 repeticiones con un cliente reutilizado y 8 creando un cliente nuevo en cada operación, igual que lo hace hoy la API.

Consulta	Plan ganador	Claves exam.	Documentos examinados	Devueltos
C1 con índice compuesto	`IXSCAN → FETCH → SORT → PROJECTION`	27	**27**	5
C1 sin índice (forzado)	`COLLSCAN → SORT → PROJECTION`	0	100	5
Órdenes «En tránsito» por fecha, con índice	`IXSCAN → FETCH` (sin `SORT`)	21	**21**	21
Ídem sin índice (forzado)	`COLLSCAN → SORT`	0	100	21
Seguimiento de una orden	`EXPRESS_IXSCAN (orden_id_1)`	1	**1**	1
C3: valoraciones ≤ 2 (sin índice)	`COLLSCAN`	0	100	37

`executionTimeMillis` fue 0–2 ms en todos los casos porque cada colección tiene solo 100 documentos. Por eso la métrica que importa es la **selectividad**: con el índice, C1 examina un 73 % menos de documentos y el listado por fecha un 79 % menos, sin ordenar en memoria. Ese ahorro crece linealmente con el volumen de órdenes.

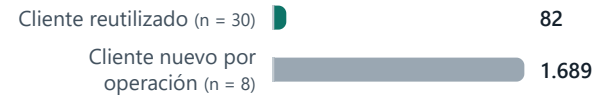
Volumen de datos retornado (C1)

Variante	Docs	Bytes	vs. base
Sin proyección ni límite	15	2.795	—
Con proyección (5 campos)	15	1.850	−34 %
Proyección + `limit(5)`	5	623	−78 %

Una orden con su seguimiento (`GET /ordenes/{id}`) pesa 552 bytes.

Latencia de una lectura por `_id`

mediana en milisegundos (menor es mejor)



Crear un `MongoClient` por operación es **unas 20 veces más lento**, porque repite la resolución DNS SRV, el TLS y la autenticación en cada llamada. Es lo que hace hoy `get_database()` en cada request, y explica el `X-Process-Time` de 1,3–1,6 s de la API (§7.2). El ping de 82 ms muestra que el resto del tiempo es red, no procesamiento del motor.

Mejoras propuestas, basadas en las mediciones

#	Mejora	Justificación con los datos medidos
1	Crear un solo `MongoClient` al iniciar la API (lifespan de FastAPI) y reutilizarlo.	Bajaría cada request de ~1,4 s a 0,1–0,3 s (1 a 3 viajes de red de ~82 ms).

#	Mejora	Justificación con los datos medidos
2	Índice `{estado: 1, total: -1}` para C1.	Por la regla ESR (igualdad, orden, rango), elimina la etapa `SORT` en memoria observada en el plan.
3	Índice `{puntaje: 1}` en `valoraciones`.	C3 hoy hace `COLLSCAN` de 100 documentos para devolver 37.
4	Mantener proyecciones y límites en los endpoints.	Reducen entre 34 % y 78 % los bytes transferidos.
5	Guardar las fechas como `Date` (ISODate) en vez de texto.	Permite rangos y `$dateDiff` directos sobre índices, sin convertir con `$dateFromString`.

5 Operaciones CRUD completas

IE7 · IE8

5.1 Diseño de las operaciones

Cada operación trabaja sobre la orden y sus documentos relacionados, para que el seguimiento y las valoraciones no pierdan su contexto. Las mismas operaciones existen como funciones en `scripts/crud_operations.py` y como endpoints en `api/main.py`.

Operación	Función / endpoint	Colecciones	Operadores	Validación incluida
Create	`crear_orden` `POST /ordenes`	ordenes_entrega + seguimiento	`insert_one` ×2	Rechaza un `_id` repetido (400). Pydantic exige los campos y `total > 0` (422).
Read	`leer_orden` `GET /ordenes/{id}`	ordenes_entrega + seguimiento	`aggregate`: `$match` + `$lookup`	404 si la orden no existe.
Update	`actualizar_estado_orden` `PUT /ordenes/{id}/estado`	ordenes_entrega + seguimiento	`update_one` con `$set` y `$push`	`matched_count == 0` → 404.
Delete	`eliminar_orden_cascada` `DELETE /ordenes/{id}`	ordenes_entrega + seguimiento + valoraciones	`delete_many` + `delete_one`	`deleted_count == 0` → 404. La cascada evita documentos huérfanos.

5.2 Ejecución del ciclo completo

Evidencia 3 · salida de `python scripts/crud_operations.py` 26-10-2026

```
--- Probando Ciclo de Vida CRUD Completo ---

 [CREATE] Orden 'orden_test_999' registrada correctamente con su seguimiento inicial.
[READ] Detalle de 'orden_test_999': Estado=En preparación | Total=$185,000.00
[UPDATE] Estado de la orden 'orden_test_999' actualizado a 'En tránsito'.
[READ] Detalle de 'orden_test_999': Estado=En tránsito | Total=$185,000.00
[DELETE] Orden 'orden_test_999' y registros relacionados eliminados.
[READ] No se encontró la orden 'orden_test_999'.
```

La lectura después de cada paso confirma el efecto: el estado cambia de «En preparación» a «En tránsito» y, tras el borrado, la orden ya no existe. Al final, la base volvió a tener 100 documentos por colección, sin restos de la prueba.

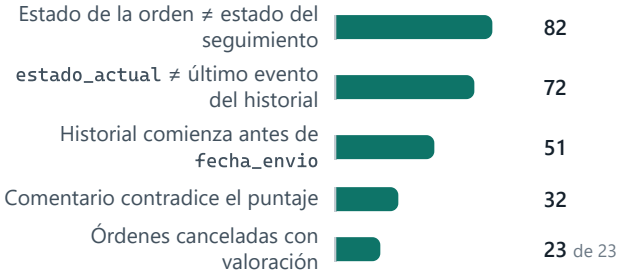
5.3 Validación de resultados: duplicados, inconsistencias y pérdida de contexto

Se ejecutaron 14 controles con agregaciones sobre la base: 9 confirman la integridad de la implementación y 5 revelan inconsistencias que ya venían en los datos de origen.

Control de integridad	Resultado	Estado
Orden con `_id` repetido	rechazada	OK
Seguimientos por orden (índice único)	1 de 1	OK
Órdenes con ruta inexistente	0	OK
Valoraciones de órdenes inexistentes	0	OK
Seguimientos huérfanos / órdenes sin seguimiento	0 / 0	OK
Restos tras el borrado en cascada	0	OK
Puntajes fuera del rango 1–5	0	OK
Clientes con correo duplicado	0	OK
Órdenes unidas a su cliente: por texto → por `cliente_num_id`	0 → 100	RESUELTO

Inconsistencias en los datos de origen

documentos afectados de 100 (23 en el último caso)



Ejemplo, `orden_2`: la orden dice «Entregado», su seguimiento dice «En preparación» y el último evento del historial es «Entregado». «Comentario contradice» = comentario positivo con puntaje ≤ 2 (23 casos) o negativo con puntaje ≥ 4 (9 casos).

```
# Detección: órdenes cuyo estado no coincide con su seguimiento  →  [{"n": 82}]
db.ordenes_entrega.aggregate([
  {"$lookup": {"from": "seguimiento", "localField": "_id", "foreignField": "orden_id", "as": "seg"}},
  {"$unwind": "$seg"},
  {"$match": {"$expr": {"$ne": ["$estado", "$seg.estado_actual"]}}},
  {"$count": "n"}])
# Detección: estado_actual distinto del último evento del historial  →  [{"n": 72}]
db.seguimiento.aggregate([
  {"$match": {"$expr": {"$ne": ["$estado_actual", {"$arrayElemAt": ["$historial.estado", -1]}]}}},
  {"$count": "n"}])
```

Soluciones aplicadas y propuestas

Problema	Solución aplicada en el proyecto	Solución propuesta
Órdenes duplicadas	Verificación previa del `_id` (400 en la API) y unicidad nativa de `_id`.	—
Más de un seguimiento por orden	Índice único `orden_id_1`.	—
Pérdida de contexto al borrar	Borrado en cascada de seguimiento y valoraciones.	Ejecutarlo dentro de una transacción.
Cliente referenciado como texto y como entero	Normalización a `cliente_num_id` en el ETL.	Aplicar la misma normalización en `POST /ordenes`, que hoy no crea ese campo.
Estados contradictorios (82 y 72 casos)	—	Una sola fuente de verdad: `estado_actual` = último evento del historial, con la orden y su seguimiento actualizados en la misma transacción. Corrección masiva con `update_many`.
Fechas incoherentes (51 casos)	—	Validar en la API que `fecha_evento ≥ fecha_envio` y agregar un validador `$jsonSchema` a la colección.
Comentario vs. puntaje (32) y valoraciones de canceladas (23)	—	Regla de negocio: solo se valoran órdenes entregadas, con comentario libre y puntaje validados por separado.

Riesgo de atomicidad detectado

Crear, actualizar y borrar una orden escribe en 2 o 3 colecciones con llamadas separadas. Si una falla a mitad de camino (por ejemplo, el `insert_one` del seguimiento), queda una orden sin seguimiento. MongoDB permite transacciones multidocumento en replica sets: `with client.start_session() as s: s.with_transaction(...)`. Así, las escrituras se confirman todas o ninguna.

Las correcciones propuestas no se ejecutaron sobre la base para conservar intacto el conjunto de datos oficial de la evaluación.

6 Integración con aplicación externa

IE9 · IE10

6.1 Flujo de integración

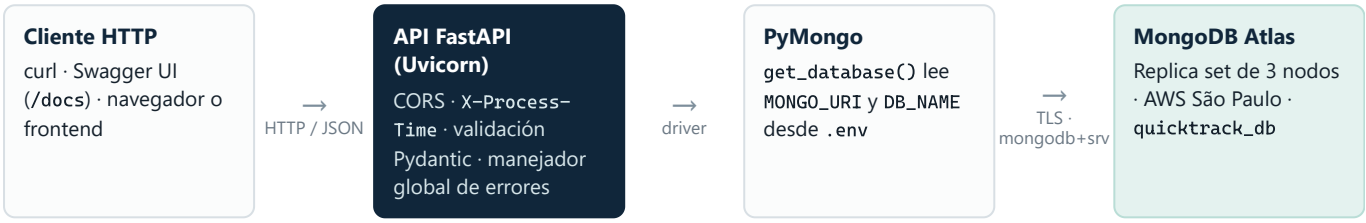


Figura 3. Cada request valida la entrada, ejecuta las operaciones en Atlas y devuelve JSON con su tiempo de proceso.

6.2 Herramientas y librerías

Componente	Versión	Rol
Python	3.14.2	Lenguaje de la aplicación
PyMongo	4.19.0	Driver oficial (SDK)
FastAPI	0.162.0	Framework de la API REST
Pydantic	2.14.1	Validación de entradas
Uvicorn	0.53.0	Servidor ASGI
python-dotenv	1.2.4	Variables de entorno
curl	8.18.0	Cliente HTTP de prueba

6.3 String de conexión

```
# .env  (excluido del repositorio por .gitignore)
MONGO_URI=mongodb+srv://<usuario>:
<contraseña>@quicktrackcluster.msimh8w.mongodb.net/
DB_NAME=quicktrack_db
PORT=8000
```

- `mongodb+srv` obtiene por DNS la lista de los 3 nodos del replica set y activa TLS por defecto.
- El código nunca contiene credenciales: `config/database.py` las lee con `load_dotenv()` y crea el cliente con `serverSelectionTimeoutMS=5000`.

6.4 Funciones expuestas por la API

Método	Ruta	Función	Respuestas
GET	`/`	Estado del servicio (healthcheck)	200
POST	`/ordenes`	Crea la orden y su seguimiento inicial	201 · 400 · 422
GET	`/ordenes/{orden_id}`	Orden + seguimiento (`$lookup`)	200 · 404
PUT	`/ordenes/{orden_id}/estado`	Cambia el estado y agrega un evento al historial	200 · 404 · 422
DELETE	`/ordenes/{orden_id}`	Borra la orden con su seguimiento y valoración	200 · 404
GET	`/analitica/rendimiento-zonas`	Órdenes y montos por zona (agregación)	200

6.5 Evidencia de lectura y escritura en tiempo real

Cada escritura hecha con **curl** a través de la API se verificó de inmediato con una lectura directa desde **otro cliente** (PyMongo). Esto confirma que los cambios quedan persistidos en Atlas apenas responde la API.

Evidencia 4 · curl → API → Atlas, verificado con PyMongo — 26-10-2026

```
$ curl -X POST /ordenes -H 'Content-Type: application/json' -d @b_ok.json
  -> HTTP/1.1 201 Created | X-Process-Time: 1.4653s
  {"mensaje":"Orden creada exitosamente","orden_id":"orden_api_901"}

$ python (PyMongo) -> db.ordenes_entrega.find_one({'_id': 'orden_api_901'})
  -> {'_id': 'orden_api_901', 'estado': 'En preparación', 'total': 125000.0} | eventos en historial: 1

$ curl -X PUT /ordenes/orden_api_901/estado -H 'Content-Type: application/json' -d @b_estado.json
  -> HTTP/1.1 200 OK | X-Process-Time: 1.3668s
  {"mensaje":"Estado de la orden 'orden_api_901' actualizado a 'En tránsito'"}

$ python (PyMongo) -> db.ordenes_entrega.find_one({'_id': 'orden_api_901'})
  -> {'_id': 'orden_api_901', 'estado': 'En tránsito', 'total': 125000.0} | eventos en historial: 2

$ curl -X DELETE /ordenes/orden_api_901
  -> HTTP/1.1 200 OK | X-Process-Time: 1.6171s
  {"mensaje":"Orden 'orden_api_901' y sus datos asociados fueron eliminados."}

$ python (PyMongo) -> db.ordenes_entrega.find_one({'_id': 'orden_api_901'})
  -> None | eventos en historial: None
```

Cuerpo enviado en el POST (`b_ok.json`): `{"_id":"orden_api_901","cliente_id":"cli_15","ruta_id":"ruta_2","medio_pago":"Tarjeta","total":125000}`. En el PUT (`b_estado.json`): `{"nuevo_estado":"En tránsito","fecha_evento":"2026-10-26T15:00:00"}`.

7 Control de errores y validaciones

IE11

7.1 Mecanismos implementados

Capa	Mecanismo	Dónde
Configuración	Si falta `MONGO_URI`, se lanza un `ValueError` con un mensaje claro.	`config/database.py`
Conectividad	`test_connection()` con `ping`, y `serverSelectionTimeoutMS=5000` para no quedar bloqueado.	`config/database.py`
Entrada	Modelos Pydantic: campos obligatorios, tipos y `total > 0`. Si algo no cumple, la API responde 422 indicando el campo y el motivo.	`api/models.py`
Reglas de negocio	Orden duplicada → 400; orden inexistente → 404 (en GET, PUT y DELETE).	`api/main.py`
Errores inesperados	Un manejador global convierte cualquier excepción en un 500 con JSON estructurado.	`api/middlewares.py`
Trazabilidad	Cabecera `X-Process-Time` en cada respuesta, más el log de acceso de Uvicorn (cliente, método, ruta y código).	`api/middlewares.py` · Uvicorn
Integridad	Índice único en `seguimiento.orden_id`, borrado en cascada y verificación de `matched_count` / `deleted_count`.	scripts y API
Scripts	`try/except` por operación, con mensajes que identifican la orden afectada.	`scripts/`

7.2 Matriz de pruebas de integración y errores

#	Caso de prueba	Esperado	Obtenido	X-Process-Time
1	`GET /` (salud del servicio)	200	**200** online	0,4 ms
2	`POST /ordenes` válida	201	**201** orden creada	1,47 s
3	`POST /ordenes` con el mismo `_id`	400	**400** «ya existe»	1,36 s
4	`POST /ordenes` con `total = -5`	422	**422** `greater_than` en `total`	1,2 ms
5	`POST /ordenes` sin `ruta_id`	422	**422** `missing` en `ruta_id`	0,5 ms
6	`GET /ordenes/orden_api_901`	200 + seguimiento	**200** orden con historial	1,36 s
7	`PUT` estado «En tránsito»	200	**200** historial: 2 eventos	1,37 s
8	`PUT` sobre una orden inexistente	404	**404** «no encontrada»	1,63 s
9	`DELETE /ordenes/orden_api_901`	200	**200** borrado en cascada	1,62 s
10	`GET` de la orden ya eliminada	404	**404** «no encontrada»	3,75 s
11	`DELETE` repetido	404	**404** «no encontrada»	1,44 s
12	`GET /analitica/rendimiento-zonas`	200	**200** 5 zonas	1,30 s
13	`GET /openapi.json`	200	**200** 5 rutas documentadas	—
14	Base inaccesible (`MONGO_URI` inválida, simulación)	500 controlado	**500** JSON estructurado	—

Los casos 4 y 5 se rechazan en menos de 2 ms porque Pydantic los valida **antes** de tocar la base. Los que consultan Atlas tardan 1,3–1,6 s por la creación del cliente en cada request (mejora 1 de §4.5).

7.3 Trazabilidad: registro de actividad y error controlado

Evidencia 5 · log de acceso de Uvicorn — extracto

```
INFO:   "POST /ordenes HTTP/1.1" 201 Created
INFO:   "POST /ordenes HTTP/1.1" 400 Bad Request
INFO:   "POST /ordenes HTTP/1.1" 422 Unprocessable Content
INFO:   "POST /ordenes HTTP/1.1" 422 Unprocessable Content
INFO:   "GET /ordenes/orden_api_901 HTTP/1.1" 200 OK
INFO:   "PUT /ordenes/orden_api_901/estado HTTP/1.1" 200 OK
INFO:   "PUT /ordenes/orden_no_existe/estado HTTP/1.1" 404 Not Found
INFO:   "DELETE /ordenes/orden_api_901 HTTP/1.1" 200 OK
INFO:   "GET /ordenes/orden_api_901 HTTP/1.1" 404 Not Found
INFO:   "GET /analitica/rendimiento-zonas HTTP/1.1" 200 OK
```

Evidencia 6 · simulación de base inaccesible con `MONGO_URI="mongodb+srv://usuario:clave@cluster-inexistente.invalid/"` — TestClient

```
GET /ordenes/orden_2 con MONGO_URI invalida -> 500
{'status': 'error', 'message': 'Error interno en el servidor QuickTrack', 'detail': 'The DNS query name does not
exist: _mongodb._tcp.cluster-inexistente.invalid.'}
```

El log registra cada llamada con su código de respuesta (se omitieron la IP y el puerto del cliente, 127.0.0.1). Ante una caída de la base, la API no se cae: responde un JSON estructurado.

7.4 Brechas detectadas y mejoras

Brecha	Riesgo	Mejora propuesta
No hay un log propio de la aplicación (solo el de acceso de Uvicorn).	No queda registro de qué documento cambió ni del error interno.	Módulo `logging` con id de request, operación, `_id` afectado y resultado, en un archivo rotativo.
El 500 devuelve `str(exc)` al cliente.	Expone detalles internos, como el host del clúster.	Guardar el detalle en el log y responder un mensaje genérico con un id de error.
`nuevo_estado` acepta cualquier texto.	Estados inválidos o mal escritos.	`Literal` / `Enum` con los 5 estados válidos (422 si no coincide).
`fecha_envio` fija (`"2026-03-30T10:00:00"`) al crear una orden.	Las órdenes nuevas quedan con una fecha que no es real.	`datetime.now(timezone.utc)` .
No se verifica que `cliente_id` y `ruta_id` existan.	Órdenes con referencias rotas.	Consulta previa a `clientes` y `rutas` (422 si no existen).
Escrituras en varias colecciones sin transacción.	Una orden puede quedar sin seguimiento.	`start_session()` + `with_transaction()` .

8 Conclusiones y próximos pasos

Conclusiones

- **Base funcional y bien configurada.** MongoDB Atlas quedó operativo en un replica set de 3 nodos en São Paulo, con credenciales fuera del código, consistencia equivalente a Session y conectividad verificada con 4 herramientas.
- **Datos cargados y verificados.** Las 5 colecciones coinciden documento a documento con sus archivos. La normalización del ETL hizo posible el cruce órdenes–clientes, que antes daba 0 de 100.
- **Consultas que responden al negocio.** Filtros, proyecciones, agregaciones de hasta tres colecciones y ordenamientos identifican a Centro como zona crítica: 47,1 % de cancelación y 2,71 de satisfacción.
- **Eficiencia medida, no supuesta.** Los índices reducen entre 73 % y 79 % los documentos examinados, y la búsqueda del seguimiento de una orden examina solo 1 documento. El cuello de botella real está en la API (un cliente nuevo por request, unas 20 veces más lento), y su solución es directa.
- **CRUD e integración en tiempo real.** 14 de 14 pruebas dieron el resultado esperado, y cada escritura por HTTP se observó de inmediato desde otro cliente.
- **Calidad de datos como hallazgo.** El 82 % de las órdenes tiene un estado distinto al de su seguimiento. Esto requiere una sola fuente de verdad para el estado antes de usar estos datos en reportes de gestión.

Indicadores propuestos para el seguimiento operativo

Indicador	Definición	Valor actual (línea base)
Tasa de cancelación por zona	Órdenes canceladas / órdenes de la zona	23,0 % global · Centro 47,1 %
Satisfacción promedio	Promedio del puntaje (1–5)	3,07 global · Centro 2,71
Valoraciones críticas	% de valoraciones con puntaje ≤ 2	37 %
Órdenes abiertas	Órdenes en preparación o en tránsito	50 de 100
Tasa de entrega	Órdenes entregadas / total	27 %
Coherencia del estado	% de órdenes cuyo estado coincide con su seguimiento	18 %
Latencia de la API (p50)	Mediana de `X-Process-Time` en endpoints que consultan la base	1,44 s (meta: < 0,3 s)

Próximos pasos priorizados

#	Acción	Impacto	Esfuerzo
1	Un solo `MongoClient` reutilizado por la API	ALTO	BAJO
2	Transacciones y una sola fuente de verdad para el estado de la orden	ALTO	MEDIO
3	Validaciones: estados con `Enum`, `$jsonSchema` y existencia de cliente y ruta	ALTO	MEDIO
4	Log estructurado y respuestas 500 sin detalles internos	MEDIO	BAJO
5	Índices `{estado, total}` y `{puntaje}`	MEDIO	BAJO
6	Fechas como `Date` y `requirements.txt` actualizado	MEDIO	BAJO

Anexo A · Estructura del repositorio y cómo reproducir

```
QuickTrack/
├── api/
│   ├── main.py          # endpoints REST
│   ├── models.py        # validación Pydantic
│   └── middlewares.py   # tiempo + error 500
├── config/
│   └── database.py      # conexión (lee .env)
├── data/                # 5 JSON del caso
├── docs/                # informe y pauta
├── scripts/
│   ├── cargar_atlas.py      # ETL
│   ├── consultas_nosql.py   # índices + C1–C3
│   └── crud_operations.py   # ciclo CRUD
├── .env.example
└── requirements.txt
```

```
# 1. Dependencias (compatibles con Python 3.14)
pip install fastapi "uvicorn[standard]" pymongo
pip install python-dotenv pandas
# 2. Copiar .env.example a .env y completar MONGO_URI
# 3. Verificar, cargar y probar
python config/database.py
python scripts/cargar_atlas.py     # borra y recarga
python scripts/consultas_nosql.py
python scripts/crud_operations.py
# 4. API → http://localhost:8000/docs
python -m uvicorn api.main:app --reload
```